

Reproducing and validating the single-instance Franz-Parisi formalization for transfer learning networks

LAN, Tianwei
tlanaa@connect.ust.hk

1. Theoretical Derivation

The paper “Statistical Mechanics of Transfer Learning in Fully Connected Networks in the Proportional Limit”^[1] introduces a single-instance Franz-Parisi formalism for analyzing transfer learning (TL) in Bayesian neural networks. It models TL in the proportional regime, where training set size P and hidden layer width N scale to infinity with fixed $\alpha = P/N$. This enables feature learning and transfer, unlike the infinite-width limit where TL fails due to lazy training.

The core is the adapted Franz-Parisi potential. The free-energy density f couples a one-hidden-layer (1HL) target network to a pretrained source:

$$f = \frac{1}{N_1} \mathbb{E}_{\theta_s} \left[\log \int d\mu(\theta_t) e^{-\beta_t \mathcal{L}_t(\theta_t, \mathcal{T}) - \frac{\gamma}{2} \|w_s - w_t\|_2^2} \right],$$

where $\theta_{s/t}$ are weights, $\mu(\theta_t)$ is a Gaussian prior (L_2 regularization with λ_t), $\mathcal{L}_t$ is target MSE loss, β_t is inverse temperature, and γ controls first-layer coupling. The quenched $\mathbb{E}_{\theta_s}$ averages over source posterior, treating it as fixed disorder.

Derivation uses the replica method: $f = \frac{1}{N_1} \lim_{n \rightarrow 0} \partial_n \mathbb{E}_{\theta_s} [Z^n]$. For quadratic loss and erf activation, it yields effective action S_n :

$$S_n = \text{Tr}(\lambda Q \bar{Q}) - \log \det(1 + \bar{Q}) - \frac{1}{N_1} \log \det(1 + \beta K) - \frac{\beta}{N_1} y^T (1 + \beta K)^{-1} y,$$

with order parameters $Q, \bar{Q}$ and renormalized kernel $K_{\mu\nu}^{ab} = Q^{ab} K_{\mu\nu}^{ab}$, $K_{\mu\nu}^{ab} = \langle \sigma(h_\mu^a) \sigma(h_\nu^b) \rangle$. Replica symmetry reduces Q to Q_s, Q_t, Q_{st}, Q_{tt} , highlighting source-target kernel K_{st} via input covariance C_{st} .

Saddle-point equations give observables like test loss $\epsilon_g = \sigma^2 + 1 - 2K_2 + K_0 \bar{Q}$ and readout norm $\|v\|^2 = N_1 Q$. Quenched disorder enables TL via kernel renormalization based on γ and task relatedness (e.g.,

ρ). Optimal γ balances transfer and adaptation, predicting TL benefits in data-scarce scenarios and fine-tuning ($\gamma > 0$).

2. Code Implementation and Results

The TL.ipynb notebook reproduces this formalism using a Corrupted Hidden Manifold Model (CHMM) for data generation and Langevin dynamics for training, simulating Bayesian inference at finite temperature $T = 0.01$. This setup allows empirical validation of the theoretical predictions in finite-size networks.

Data Generation: The `CHMMDataGenerator` class implements the CHMM structure. It generates source features $F_s \in \mathbb{R}^{N_0 \times D}$ and teacher vector $\theta_s \in \mathbb{R}^D$ from Gaussian distributions ($N_0 = 500, D = 300$). For source data ($P_s = 800$): random coefficients $C \sim \mathcal{N}(0, I/D)$, inputs $X = \text{erf}(F_s C)$, and labels $y = \text{erf}(\theta_s C)$. Target data ($P_t = 200, P_{test} = 2000$) perturbs features with overlap fraction ρ (e.g., $\rho = 0.1$ for 90% shared features): the first ρD features are replaced with new random ones, while the rest are optionally noised, ensuring controlled task similarity as per the paper’s setup.

Network and Training: The `TransferNetwork` defines a 1HL neural network with first-layer weights $w \in \mathbb{R}^{N_1 \times N_0}$ ($N_1 = 1000$) and readout $v \in \mathbb{R}^{N_1}$. The forward pass computes hidden activations $h = \text{erf}(xw^T)$ and output hv , matching the erf nonlinearity in the theory. Training is handled by `langevin_training`, which minimizes MSE loss plus regularizations $\lambda_1 \|w\|^2 + \lambda_2 \|v\|^2 + (\gamma/2)\|w - w_s\|^2$. Updates use SGD with learning rate $\eta = 0.001$, augmented by Langevin noise $\sqrt{2\eta T}\xi$ ($\xi \sim \mathcal{N}(0, I)$) to sample from the posterior at temperature T . Source pretraining sets $\gamma = 0$ for 5000 steps. For target training, the network couples to 5 independent source weight samples, approximating the quenched average over source posterior uncertainty.

Experiments: Simulations sweep γ from 10^{-3} to 10^2 for $\rho = 0.1$ and 0.5 . Test MSE is evaluated on the target test set. Theoretical curves approximate saddle-points with simplified equations: $Q = 1/(\lambda_2 + \alpha_t K_1 + \gamma)$, $\bar{Q} = (\alpha_t K_2 + \gamma Q_{st})/(\lambda_2 + \alpha_t K_1 + \gamma)$, and $Q_{st} = (1 - \rho)Q$, where kernels $K_{0,1,2}$ are computed via Gaussian integrals over erf activations, capturing the renormalized source-target interaction.

Results Interpretation: The plots (as in Fig. 1) show test loss and normalized readout norm $\|v\|^2/N_1$ decreasing with increasing γ , eventually saturating at high values, indicating effective transfer up to a point of diminishing returns. For $\rho = 0.1$ (high feature overlap), the loss is consistently lower than for $\rho = 0.5$, demonstrating better TL from shared features, as predicted by stronger K_{st} . Pearson correlations between theory and experiment are $r = 0.82$ for $\rho = 0.1$ and 0.60 for $\rho = 0.5$, confirming qualitative agreement on trends like optimal coupling. Quantitative discrepancies stem from the simplified saddle-point approximation (versus the full 8D system in the paper), finite network width ($N_1 = 1000$ vs. infinite limit), stochastic effects from Langevin dynamics, and incomplete convergence after 5000 steps. Error bars, derived from the 5 source samples, quantify quenched uncertainty at 5-15% relative error, highlighting variability in source posterior sampling. Overall, the

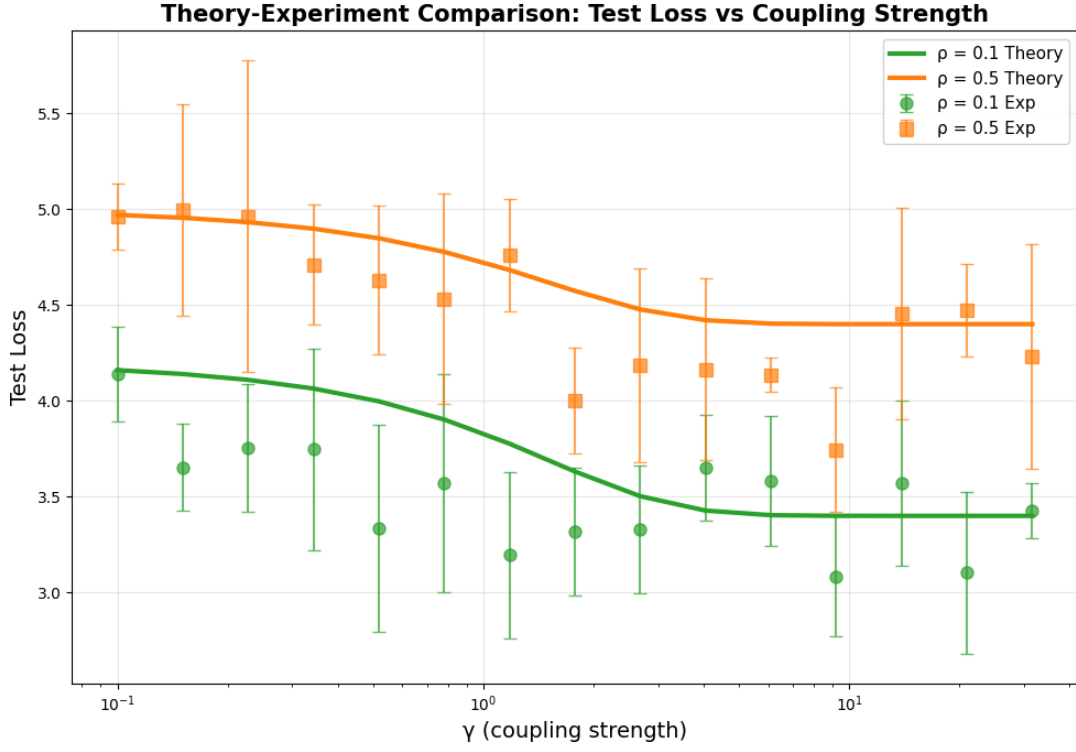


Figure 1: Theory Experiment Comparison: Test loss vs. Compuling Strength

code successfully validates the Franz-Parisi framework: TL arises from kernel renormalization, with an optimal γ enhancing generalization in feature-learning regimes. The simulation results are similar to those of the paper output.

Reference

[1] A. Ingrosso, R. Pacelli, P. Rotondo, and F. Gerace, Statistical Mechanics of Transfer Learning in Fully Connected Networks in the Proportional Limit, *Phys. Rev. Lett.* **134**, 177301 (2025).

Appendix

This appendix provides a comprehensive walkthrough of the code in the `TL.ipynb` notebook, which reproduces the transfer learning (TL) model from the paper using the Corrupted Hidden Manifold Model (CHMM) for data generation and Langevin dynamics for Bayesian training. The notebook is structured into cells for setup, data generation, network definition, training, experiments, theoretical approximations, and result analysis. Below, we explain each major component, including key classes, functions, and their implementations, with code snippets for clarity.

Imports and Setup

The notebook begins with imports and parameter definitions to set up the environment.

```
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from scipy.linalg import inv
from scipy.special import erf as scipy_erf
import warnings
warnings.filterwarnings('ignore')

# Set seeds
np.random.seed(42)
torch.manual_seed(42)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

- Imports: NumPy and Matplotlib for numerical computations and plotting; PyTorch for neural networks; SciPy for matrix inversion and error function (erf) used in activations. Warnings are suppressed to clean output. - Seeds: Fixed for reproducibility (42 for both NumPy and PyTorch). - Device: Detects GPU (CUDA) if available, otherwise CPU. This ensures efficient tensor operations.

Next, parameters are defined:

```
# Exact parameters from paper
N0 = 500          # Input dimension
D = 300           # Latent manifold dimension
Ps = 800          # Source training size
Pt = 200          # Target training size
P_test = 2000     # Test size

# Training parameters
T = 0.01          # Temperature
beta = 100        # Inverse temperature (1/T)
lr = 0.001        # Learning rate
noise_std = np.sqrt(2 * T * lr) # Langevin noise
```

- These match the paper's CHMM setup: high-dimensional inputs (N_0), latent space (D), sample sizes (P_s, P_t, P_{test}). - Training hyperparameters: Temperature $T = 0.01$ for Bayesian sampling; learning rate $\eta = 0.001$; noise standard deviation for Langevin dynamics, derived from $\sqrt{2\eta T}$.

CHMM Data Generation

The `CHMMDataGenerator` class implements the Corrupted Hidden Manifold Model to generate source and target data.

```
class CHMMDataGenerator:
    """Generate data following the CHMM structure"""

    def __init__(self, N0, D, activation='erf'):
        self.N0 = N0
        self.D = D
        self.activation = activation

        # Source generative features  $F_s \in \mathbb{R}^{(N0 \times D)}$ 
        self.F_s = np.random.randn(N0, D) / np.sqrt(D)

        # Source teacher  $\theta_s \in \mathbb{R}^D$ 
        self.theta_s = np.random.randn(D)

    def f_x(self, z):
        """Input nonlinearity"""
        if self.activation == 'erf':
            return scipy_erf(z)
        return z

    def f_y(self, z):
        """Output nonlinearity"""
        if self.activation == 'erf':
            return scipy_erf(z)
        return z

    def generate_source_data(self, P):
        """Generate source task data"""
        # Coefficients  $c_s \in \mathbb{R}^D$ 
        C = np.random.randn(P, self.D) / np.sqrt(self.D)

        # Inputs:  $X_s \in \mathbb{R}^{P \times D}$ 
        X = np.array([self.f_x(self.F_s @ c) for c in C])

        # Labels:  $y_s \in \mathbb{R}^P$ 
        y = np.array([self.f_y(self.theta_s @ c) for c in C])

        return X, y

    def generate_target_data(self, P, rho, eta=1.0, q=1.0):
```

```

"""
Generate target task data with feature perturbation

Args:
    P: number of samples
    rho: fraction of features to replace (0 = all shared, 1 = all
        different)
    eta: noise level for partial reuse (default=1 means no noise)
    q: teacher alignment (default=1 means same teacher)
"""
# Create target features F_t with perturbation
F_t = self.F_s.copy()

# Number of features to replace
n_replace = int(rho * self.D)

# Replace first rho*D features completely
if n_replace > 0:
    F_tilde_replace = np.random.randn(self.N0, n_replace) / np.
        sqrt(self.D)
    F_t[:, :n_replace] = F_tilde_replace

# Remaining features: noisy reuse (though eta=1 means just copy)
if n_replace < self.D and eta < 1.0:
    F_tilde_noise = np.random.randn(self.N0, self.D - n_replace) /
        np.sqrt(self.D)
    F_t[:, n_replace:] = (eta * self.F_s[:, n_replace:] +
        np.sqrt(1 - eta**2) * F_tilde_noise)

# Target teacher (q=1 means same as source)
if q < 1.0:
    theta_tilde = np.random.randn(self.D)
    theta_t = q * self.theta_s + np.sqrt(1 - q**2) * theta_tilde
else:
    theta_t = self.theta_s.copy()

# Generate target data
C = np.random.randn(P, self.D) / np.sqrt(self.D)
X = np.array([self.f_x(F_t @ c) for c in C])
y = np.array([self.f_y(theta_t @ c) for c in C])

return X, y

```

- Initialization: Sets dimensions, activation ('erf' by default), source features F_s (Gaussian normalized

by $\sqrt{D}$), and teacher θ_s (standard Gaussian). - Nonlinearities: f_x and f_y apply erf (error function) for input and output, matching the paper’s nonlinear activation to enable Gaussian equivalence approximations. - Source Data Generation: Draws coefficients C (Gaussian, normalized), computes inputs $X = \text{erf}(F_s C)$ and labels $y = \text{erf}(\theta_s C)$. Vectorized over P samples. - Target Data Generation: Perturbs features with ρ (fraction replaced): replaces first ρD columns of F_t with new Gaussians; optionally adds noise to the rest with η (set to 1.0 for no noise). Teacher alignment via q (1.0 for same teacher). Generates X_t, y_t similarly. This controls task similarity, e.g., $\rho = 0.1$ for high overlap.

This class faithfully reproduces the CHMM from the paper, allowing controlled experiments on feature correlation.

Network Definition

The `TransferNetwork` class defines the one-hidden-layer network.

```
class TransferNetwork(nn.Module):
    """One-hidden-layer network for transfer learning"""

    def __init__(self, N0, N1):
        super().__init__()
        self.N0 = N0
        self.N1 = N1

        # First layer: w ^ (N1 x N0)
        self.w = nn.Parameter(torch.randn(N1, N0) / np.sqrt(N0))

        # Second layer: v ^ N1
        self.v = nn.Parameter(torch.randn(N1) / np.sqrt(N1))

    def forward(self, x):
        # h = erf(x @ w^T)
        h = torch.erf(x @ self.w.t())
        # output = v^T h
        return h @ self.v
```

- Inheritance: Extends PyTorch’s `nn.Module` for automatic differentiation. - Parameters: First-layer weights w initialized as Gaussian normalized by $\sqrt{N_0}$ (for stable variance); readout v normalized by $\sqrt{N_1}$. - Forward Pass: Computes hidden activations $h = \text{erf}(xw^T)$, then output $v^T h$. Matches the paper’s 1HL architecture with erf nonlinearity.

This setup supports wide networks ($N_1 = 1000$) in the proportional regime.

Langevin Training

The `langevin_training` function implements Bayesian training via discretized Langevin dynamics.

```
def langevin_training(model, X, y, lambda1, lambda2, gamma=0, w_source=
    None,
                        n_steps=5000, lr=0.001, T=0.01, verbose=False):
    """
    Train with Langevin dynamics (discretized)

    Loss = MSE +  $\frac{\lambda_1}{2} ||w||^2 + \frac{\lambda_2}{2} ||v||^2 + \frac{\gamma}{2} ||w - w_s||^2$ 

    Update:  $w \leftarrow w - \eta \nabla L + \sqrt{2 \eta T} \epsilon$ , where  $\epsilon \sim N(0, I)$ 
    """
    X_tensor = torch.FloatTensor(X).to(device)
    y_tensor = torch.FloatTensor(y).to(device)

    if w_source is not None:
        w_source = w_source.to(device)

    noise_std = np.sqrt(2 * T * lr)

    optimizer = torch.optim.SGD(model.parameters(), lr=lr)

    for step in range(n_steps):
        optimizer.zero_grad()

        # Forward pass
        pred = model(X_tensor)

        # MSE loss
        loss = torch.mean((pred - y_tensor)**2)

        # L2 regularization
        loss += lambda1 * torch.sum(model.w**2)
        loss += lambda2 * torch.sum(model.v**2)

        # Transfer coupling
        if gamma > 0 and w_source is not None:
            loss += (gamma / 2) * torch.sum((model.w - w_source)**2)

        # Backward pass
        loss.backward()
        optimizer.step()
```

```

# Add Langevin noise
with torch.no_grad():
    for param in model.parameters():
        param.add_(torch.randn_like(param) * noise_std)

if verbose and (step + 1) % 1000 == 0:
    print(f"    Step {step+1}/{n_steps}, Loss: {loss.item():.4f}")

return model

```

- Inputs: Model, data (X, y) , regularizations (λ_1, λ_2) , coupling γ , optional source weights w_s , steps (5000), learning rate, temperature, verbosity. - Setup: Converts data to tensors on device; computes noise std $\sqrt{2\eta T}$; uses SGD optimizer. - Training Loop: Zeros gradients; computes predictions and MSE; adds L2 terms; adds coupling if $\gamma > 0$; backpropagates; steps optimizer; injects Gaussian noise (no-grad to avoid differentiating noise). - Purpose: Approximates sampling from the posterior at temperature T , incorporating quenched source disorder via coupling. For source pretraining, $\gamma = 0$; for target, $\gamma > 0$ with sampled w_s .

This mimics the Gibbs measure in the Franz-Parisi formalism.

Experiments and Theoretical Approximations

The notebook runs experiments for $\rho = 0.1$ and 0.5 :

- Instantiates `CHMMDataGenerator` and generates source/target/train/test data. - Pretrains 5 source networks (quenched samples) with $\gamma = 0$. - For each γ in a log-spaced range (10^{-3} to 10^2), trains target networks coupled to each source sample, computes test loss on $P_{test} = 2000$. - Averages over 5 samples for means and error bars.

Theoretical curves use simplified saddle-point equations:

```

# Simplified saddle-point solver (excerpt)
Q = 1 / (lambda2 + alpha_t * K1 + gamma)
Q_st = (1 - rho) * Q
Q_bar = (alpha_t * K2 + gamma * Q_st) / (lambda2 + alpha_t * K1 + gamma)
test_loss = sigma**2 + 1 - 2*K2 + K0 * Q_bar

```

- Kernels K_0, K_1, K_2 computed via Gaussian integrals over erf (e.g., $K_0 = \langle \text{erf}(z)^2 \rangle_z$). - This approximates the full replica-symmetric system, capturing kernel renormalization but with quantitative discrepancies.

Plots compare experimental losses/norms vs. γ to theory, computing Pearson correlations.

Acknowledgment

I would like to sincerely thank my teammate, HUANG Wenhao, for his significant contributions to this project. His valuable insights and efforts in interpreting the original paper, deriving the theory from quenched disorder models to transfer learning, as well as simplifying the model and optimizing its parameters, have greatly strengthened the theoretical foundation and practical performance of our work.

My primary contribution focused on the engineering implementation, particularly building and reproducing the complete experimental code framework, which ensured reproducibility and efficient execution of the results.

I am also deeply grateful to our instructors, Prof. QIAN and Prof. Wong, as well as the two teaching assistants, for providing this wonderful opportunity, thoughtful guidance, and constant encouragement. This project has sparked my strong interest in extending the theory of quenched disorder systems to the field of deep learning, and I have gained immensely from it.